

Industrial Automation & Control — Week 7 Notes + Detailed Solutions

Lessons 25–29: More RLL Elements, RLL Syntax, Structured Design Approach to Sequence Control

Exam-Focused Notes from Scratch + Step-by-Step Assignment 7 Solutions

Built from your topic list and the uploaded Week 7 assignment PDF.

1. Big picture: what these lessons are really about

- **Earlier PLC lessons** — showed basic contacts, coils, scan cycle, and small ladder programs.
- **These lessons** — add more powerful elements like timers, counters, data movement, sequencing methods, and a structured design workflow.
- **Core idea** — industrial sequence control is easiest when you stop writing random ladder rungs and instead first think in terms of states, transitions, events, and actions.
- **Final goal** — convert a verbal process description into a clean model such as a finite state machine or Sequential Function Chart, then implement it in RLL.

Exam tip: Most exam questions here are not about memorizing symbols alone. They test whether you can read the process flow correctly and map it into states, timers, counters, and ladder logic.

2. RLL from scratch

2.1 What is RLL?

RLL means Relay Ladder Logic. It is a graphical programming style for PLCs that looks like old relay circuits. The left and right vertical lines act like power rails, and each rung represents a logical condition that decides whether an output, internal relay, timer, or counter should energize.

- **NO contact** — normally open contact; conducts when its referenced bit is ON/true.
- **NC contact** — normally closed contact; conducts when its referenced bit is OFF/false.
- **Coil** — output instruction; becomes ON when the rung condition is true.
- **Internal relay / memory bit** — used to store logic states without needing a physical output.
- **Series contacts** — represent logical AND.
- **Parallel contacts** — represent logical OR.

2.2 How to read one rung

1. Start at the left rail.
2. Move left to right through all contacts.
3. A NO contact passes logic when its bit is 1.
4. An NC contact passes logic when its bit is 0.
5. If at least one complete conducting path reaches the coil, the coil turns ON.
6. If no complete path exists, the coil stays OFF.

3. More RLL elements

3.1 On-delay timer (TON)

A TON waits for a preset time after its input becomes true, and only then its timer-done output turns ON.

- **Input goes ON** — timer starts counting.
- **Before preset time** — timer output remains OFF.

- **After preset time** — timer output becomes ON.
- **If input goes OFF before completion** — the timer usually resets in a non-retentive TON.

Exam tip: TON = delay before turning ON.

3.2 Off-delay timer (TOF)

An off-delay timer keeps the output ON for some time after the input goes OFF.

- **Input ON** — output turns ON immediately.
- **Input OFF** — timer starts.
- **During delay period** — output remains ON.
- **After preset time** — output turns OFF.

Exam tip: TOF = delay before turning OFF.

3.3 Pulse timer / one-shot type timed output

A pulse timer generates an output pulse of fixed duration when triggered. In timing-waveform questions, identify whether the output is delayed at the start, delayed at the end, or forced to exist only for a pulse width.

- **Pulse timer** — output is a timed pulse.
- **On-delay** — output starts late.
- **Off-delay** — output ends late.

3.4 Counter

A counter increments its count when it detects input events, usually pulses or edges. When the accumulated count reaches the preset value, the counter done bit turns ON.

- **Preset** — target count value.
- **Accumulated value** — current count reached so far.
- **Done bit** — turns ON when accumulated count reaches preset.
- **Reset** — clears accumulated count.

Exam tip: Counters react to event occurrences, not to how long an input remains high.

3.5 Data move

A data move instruction copies a value from one register/location to another. It is used when a sequence step must transfer setpoints, modes, limits, recipe values, or state numbers into working memory.

- **Move** — copy source data into destination.
- **Typical use** — load a new target value when a machine enters a new state.
- **Important** — it does not itself make a sequence happen; it supports the sequence.

3.6 Macro operation

A macro operation means grouping several repeated actions into one logical block or routine. In structured PLC design, macros improve reuse, readability, and maintenance.

- **Example** — a bottle-filling subroutine reused for several stations.
- **Advantage** — less duplicate code, easier debugging.

4. Syntax logic you must know

- **AND** — series path.

- **OR** — parallel path.
- **NOT** — NC contact or negated condition.
- **Latch / seal-in** — an output contact of the same coil is placed in parallel with the start condition to hold the coil ON.
- **Interlock** — logic that prevents two conflicting outputs from being ON together.
- **Manual override** — operator can force or jog selected actions, but safety interlocks must still dominate.

5. Sequence control from scratch

Sequence control is used when the plant operates through a logical order of steps: start -> do action A -> wait for condition -> do action B -> wait for timer -> do action C, and so on.

- **Sequence control** — best for machines, conveyors, traffic lights, filling systems, packaging, batch steps, and startup/shutdown sequences.
- **Analog control** — best for continuous variables like temperature, pressure, level, flow, or speed.
- **Discrete-event behavior** — machine changes state when events happen, such as a switch closing, a sensor triggering, or a timer completing.
- **Continuous-variable behavior** — quantities vary continuously with time and are often controlled by PID.

Exam tip: If the problem talks about order of actions, waiting conditions, or steps, think sequence control. If it talks about maintaining a variable at a value, think analog control.

6. Structured design approach to sequence control

6.1 Why structured design is necessary

- **Industrial sequence logic gets large quickly** — directly connecting inputs to outputs becomes unreliable in large systems.
- **State matters** — the same input can require different outputs depending on the current step of operation.
- **Programming errors are costly** — bad logic can damage equipment, stop production, or create safety hazards.

6.2 Standard design workflow

7. **Read the process description carefully.** identify what the machine is supposed to do, in what order, and under what conditions.
8. **Identify all inputs.** push-buttons, selectors, limit switches, sensors, timer done bits, counter done bits, fault contacts.
9. **Identify all outputs.** motors, valves, lamps, solenoids, alarms, internal state bits.
10. **Identify operating modes.** auto, manual, jog, maintenance, reset, emergency stop.
11. **Identify actions in each mode.** what must turn ON or OFF in each condition.
12. **Identify states/steps.** each distinct operating condition becomes a state.
13. **Identify transition conditions.** the event or condition that moves the system from one state to the next.
14. **Draw FSM or state diagram.** show states as nodes and transitions as arrows.
15. **Convert to Sequential Function Chart.** steps + transitions + actions.
16. **Implement in RLL.** use memory bits for states, timers/counters for delays/events, and output rungs driven by active states.
17. **Check failure cases.** sensor failure, stuck actuator, timeout, manual stop, restart behavior.
18. **Add safety logic.** E-stop, guard door, overload trip, interlocks must override normal control.

Exam tip: In exams, when asked to design from description, do not jump to ladder immediately. First list inputs, outputs, states, and transitions.

7. State, FSM, and SFC

7.1 What is a state?

A state is a condition in which the system waits or performs a defined set of actions. Example for traffic lights: Red only, Red+Amber, Green only, Amber only.

7.2 FSM idea

Finite State Machine means the system is allowed to be in one of a finite number of states. Inputs and events decide transitions between states.

- **State** — what the system is currently doing.
- **Transition** — rule for moving to the next state.
- **Action** — output(s) active while the state is active.

7.3 Sequential Function Chart (SFC)

SFC is a formal way to represent sequence logic. It shows steps (states), actions associated with steps, and transitions between steps.

- **Initial step** — the step active after power-on or reset.
- **Transition** — must be true for the active step to hand over control to the next step.
- **Alternative branches** — one branch out of many is chosen.
- **Parallel branches** — multiple branches execute concurrently.
- **Rejoin** — parallel branches synchronize later.

Exam tip: SFC makes complex sequences easier to understand, test, and sometimes auto-generate into ladder logic.

8. Parallel branch in SFC

A parallel branch means two or more sequences execute at the same time. This is useful when two machine actions can proceed concurrently.

- **Parallel split** — one active step activates multiple branches.
- **Parallel join** — branches must usually complete before the sequence continues.
- **Illegal practice** — jumping arbitrarily into or out of a parallel branch creates undefined behavior.

9. Manual control, safety, and failure handling

- **Manual mode** — lets operator directly command outputs or jog actuators.
- **Auto mode** — sequence advances automatically based on events.
- **Transient forms** — consider what happens during startup, reset, power restoration, or changing modes.
- **Possible failure** — sensor stuck ON/OFF, actuator not responding, timeout, counter not resetting, broken interlock.
- **Safety rule** — manual mode must not bypass E-stop, overload trip, or dangerous interlocks.
- **Practical design** — include reset path, alarm path, and fail-safe behavior.

10. How to convert a word problem into logic

19. **Underline nouns.** they often become inputs, outputs, or internal states.
20. **Underline action words.** start, stop, open, close, wait, count, fill, move, alarm.
21. **Mark time-related phrases.** after 20 s, for 2 min, delay, until.
22. **Mark event-related phrases.** when sensor becomes true, after 10 bottles, if guard opens.
23. **Separate outputs from conditions.** lamp ON is an action; sensor X true is a condition.
24. **Write state names.** State 0, State 1, State 2, etc., with clear action for each.
25. **Write transition condition under each state arrow.** timer done, sensor true, count reached, push-button pressed.
26. **Only after that.** draw RLL using state bits, timers, counters, and output coils.

Exam tip: The biggest mistake is mixing state memory and output logic. First create state memory. Then drive

outputs from states.

11. Typical exam-ready descriptive answers

What is an on-delay timer?

An on-delay timer delays the turning ON of its output. When the input becomes true, the timer starts counting. Only after the preset delay elapses does the timer output become true. If the input goes false before completion, a non-retentive on-delay timer resets.

What is an off-delay timer?

An off-delay timer delays the turning OFF of its output. The output turns ON immediately when the input becomes true. When the input goes false, the timer starts and the output remains ON for the preset period before turning OFF.

What is the role of a counter in PLC sequence control?

A counter stores the number of input events or pulses. When the accumulated count reaches a preset value, its done bit becomes true. It is useful in packaging, batching, bottle counting, and sequence advancement based on number of occurrences.

Why is a structured design approach important?

Because large industrial programs depend on present state and sequence history. Writing output logic directly from inputs becomes unreliable and unsafe. A structured design approach reduces errors, improves testing, supports formal verification, and avoids expensive or dangerous mistakes.

What is Sequential Function Chart?

Sequential Function Chart is a formal method for representing sequence control. It consists of steps, transitions, and actions. It helps translate process descriptions into a structured control design and then into PLC ladder logic.

Detailed Step-by-Step Solutions for Assignment 7

Q1. Match timer types with plots

Accepted answer is d, so A-II, B-III, C-I. Reasoning: In an on-delay timer, output rises only after a delay from the input turning ON, so it matches plot II. In an off-delay timer, output stays ON after the input falls, so it matches plot III. The remaining plot I is the pulse timer because it gives a fixed pulse output.

- 27. Identify whether the output is delayed at the start, delayed at the end, or exists only as a pulse.
- 28. Plot II shows delayed start -> on-delay.
- 29. Plot III shows delayed end -> off-delay.
- 30. Plot I therefore corresponds to pulse timer.

Q2. Timer 0 / Timer 1 / Lamp ladder

Accepted answer is c: Lamp is only on when Timer 0 is on. The third rung contains a Timer 0 contact directly driving the Lamp coil. Therefore the lamp follows Timer 0 state. Timer 1 appears in upper rungs for timer generation, not as the final direct condition for the lamp.

- 31. Read the rung that contains the Lamp coil, not just the timer rungs above.
- 32. The Lamp rung has Timer 0 contact in series with the coil.
- 33. Hence Lamp ON $\Leftrightarrow$ Timer 0 contact true.

Q3. Traffic light sequence chart to ladder

Accepted answer is a. The SFC says sequence is Red only -> Red+Amber -> Green only -> Amber only -> repeat, with 2 min, 20 s, 2 min, 20 s timing. Option a is the one whose state/output arrangement follows that exact order.

34. List required states first: State 0=Red, State 1=Red+Amber, State 2=Green, State 3=Amber.

35. Check each option for the order of outputs and timer durations.

36. Option a matches the required action sequence and timing progression.

Q4. Counter C460, M100, M101, Y430

Accepted answer is a: both statements are true. M100 and M101 are energized before the counting pulse sequence is applied. The counter preset is 8, so after the eighth pulse of X400 the counter done condition becomes true and Y430 switches OFF according to the rung logic involving C460 and the normally closed M100 contact in the final rung.

37. See first two rungs: M100 and M101 are prepared before counting action.

38. Counter C460 has preset 8, with X400/X401 logic driving count path.

39. After the eighth pulse, C460 done bit changes rung truth, causing Y430 to switch OFF.

Q5. Output OP02 every time

Accepted answer is b: statement i is false but ii is true. In the diagram, IN02 is the input that feeds the counter path leading to OP01. OP02 is driven by OP01 contact, so OP02 occurs when the counter reaches preset through IN02 events, not through IN01 events.

40. Check which input is connected to the counter increment logic.

41. IN02 is on the count path; IN01 is not the effective count-trigger path here.

42. When counter reaches preset 5, OP01 turns ON.

43. OP02 is energized by OP01 contact, therefore tied to IN02 count completion.

Q6. Bottle packing ladder

Accepted answer is c. We need: conveyor runs continuously in auto, photocell IN01 generates pulses when bottles pass, counter counts bottles, and solenoid OP01 turns ON when count reaches 10 and also resets the counter. Option c is the ladder that correctly combines count input and reset/solenoid behavior.

44. Translate the story into functions: count bottles, stop/change box after 10, reset counter for next batch.

45. Look for a rung where IN01 gives count pulses.

46. Look for OP01 linked to counter done condition and reset path.

47. Option c is the only arrangement consistent with the story.

Q7. Why a systematic design process is emphasized

Accepted answer is d. Industrial programming errors are expensive and can even cost human lives. So systematic design is not just about convenience; it is about safety, reliability, and maintainability.

48. In real plants, wrong logic can cause collisions, spills, overheating, crushing hazards, or costly downtime.

49. Therefore industry insists on formal design flow and thorough testing.

Q8. Automatic generation of RLL will be error-free only if

Accepted answer is c. Automatic generation is trustworthy only when the formal specifications and models are correct. Garbage in -> garbage out.

50. Program generation tool can only follow the model it is given.

51. If state model or specifications are wrong, generated ladder will also be wrong.

Q9. Why directly programming outputs from inputs becomes unreliable

Accepted answer is c. Large systems are state-dependent and inputs change too frequently; direct input-output programming ignores sequence memory and operating context.

- 52. In large systems, the same input may mean different things in different states.
- 53. Therefore state memory must be used instead of naive direct mapping.

Q10. Initial step in SFC is executed

Accepted answer is a: after power-on and after program reset. The initial step is the defined starting condition of the sequence.

- 54. Initial step is not only at first power-on.
- 55. A reset also returns the SFC to its initial step.

Q11. Two alternative transitions true simultaneously in an SFC

Accepted answer is d: the leftmost transition is selected. In standard resolution rule used here, simultaneous alternative transitions are resolved by priority, commonly left-to-right.

- 56. Since both are alternatives, both do not execute together.
- 57. A deterministic priority rule is required.
- 58. Here the leftmost branch has priority.

Q12. Illegal in Sequential Function Charts

Accepted answer is b: jumping into or out of a parallel sequence is illegal. It breaks the synchronization logic of the parallel branch structure.

- 59. Nested parallel branches may be allowed in proper form.
- 60. But arbitrary jumping into/out of parallel structure destroys sequence consistency, so it is illegal.

Ultra-Short Last-Day Revision Sheet

- **TON** — delays ON.
- **TOF** — delays OFF.
- **Counter** — counts events until preset.
- **State** — current step of machine behavior.
- **Transition** — condition to move to next state.
- **SFC** — formal sequence model: steps + transitions + actions.
- **Parallel branch** — two branches active concurrently.
- **Structured design** — Inputs -> Outputs -> Modes -> States -> Transitions -> SFC/FSM -> RLL.
- **Golden rule** — do not code outputs directly from inputs in large state-based systems.
- **Safety** — manual mode never overrides E-stop and critical interlocks.

Reference used

The uploaded Week 7 assignment PDF was used as a reference for exam focus, especially for timer waveform matching, timer-contact behavior, traffic-light sequencing, counters, structured design questions, and SFC rules.